Artificial Intelligence

Beyond Classical Search

Local beam search

- Variant of Best First Search
- Takes care of space
- Beam width is given
- Based on the beam width, the nodes are stored in the priority queue

- In Best First Search the beam width is infinite (no pruning of nodes)
- In Hill Climbing beam-width is 1

Local beam search

- Keep track of *k* states rather than just one
  - Start with *k* randomly generated states
  - At each iteration, all the successors of all *k* states are generated
  - If any one is a goal state, stop; else select the *k* best successors from the complete list and repeat.

Local Beam Search

- Local beam search can suffer from a lack of diversity among the k states—they can quickly become concentrated in a small region of the state space, making the search little more than an expensive version of hill climbing.

- Complete: NO, because might lead to a dead end
- Optimal: NO, may provide good solution, but not guaranteed to provide always an optimal one

Stochastic Beam Search

- Stochastic Beam Search: It chooses k successors at random, with the probability of choosing a given successor being an increasing function of its value.

- In stochastic beam search, the algorithm does not simply choose the k best successors, instead, *it assigns a probability to each successor*, with the probability being proportional to the successor's evaluation score.
- For example, if there are n successors, and the evaluation scores of the successors are [s1, s2, ..., sn], then the probability of selecting successor *i* is proportional to s_i.

- To calculate the probability of each successor being selected, we first need to calculate the sum of all the evaluation scores:
- sum(scores) = 3 + 7 + 5 + 2 + 8 = 25
- We can then calculate the probability of each successor being selected by dividing its evaluation score by the sum of all the scores:
  - Probability of selecting successor 1 = 3/25 = 0.12
  - Probability of selecting successor 2 = 7/25 = 0.28
  - Probability of selecting successor 3 = 5/25 = 0.20
  - Probability of selecting successor 4 = 2/25 = 0.08
  - Probability of selecting successor 5 = 8/25 = 0.32

- Nodes generate successors based on a heuristic/evaluation function (fitness function)

Genetic algorithms

- A genetic algorithm is a variant of stochastic beam search in which successor states are generated by combining two parent states rather than by modifying a single state.
- A successor state is generated by combining two parent states
- Start with *k* randomly generated states (population)
- A state is represented as a string over a finite alphabet (often a string of 0s and 1s)
- Evaluation function (fitness function). Higher values for better states.
- Produce the next generation of states by selection, *crossover*, and *mutation* (*genetic operators*)

Biology Concepts

- Population
- Fitness
- Selection
- Crossover
- Mutation

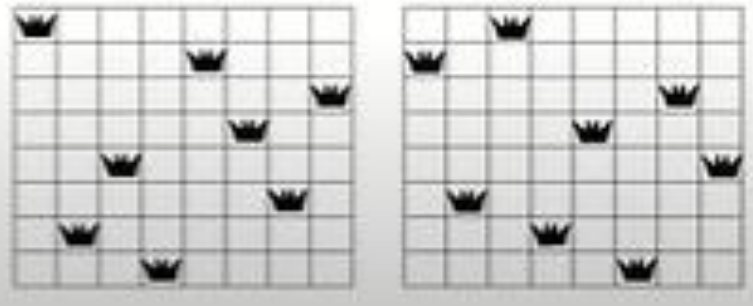
Biology Concepts: Population

Biology

- Collection of individuals.

Algorithm

- Collection of states.



Biology Concepts: Fitness

Biology

- More healthy, less prone to diseases.

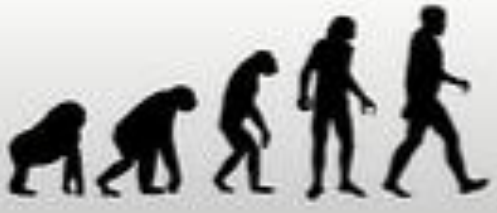
Algorithm

- Closest to the final solution.

Biology Concepts: Selection

Biology

- Selecting species that are the most biologically fit.



Algorithm

- Selecting states that are closest to the solution (Fittest).

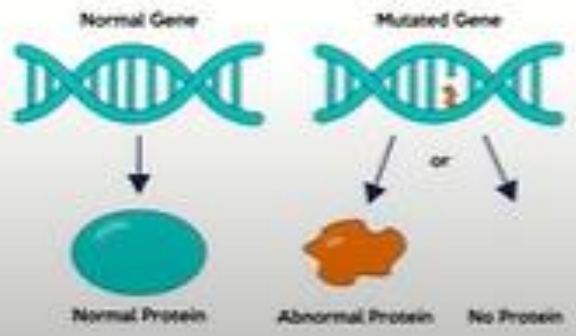
Biology Concepts: Crossover

Biology	Algorithm
generation	Interchanging values between selected states

Biology Concepts: Mutation

Biology

- Change or variation.



Algorithm

- Alteration.

Before Mutation

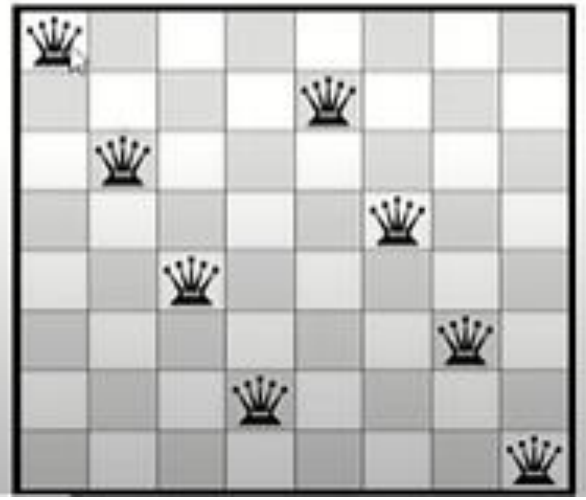
A5	1	1	1	0	0	0

After Mutation

A5	1	1	0	1	1	0

8- Queens Problem

- Arrange 8 queens on a standard chess board in such a way that no queen attacks each other.



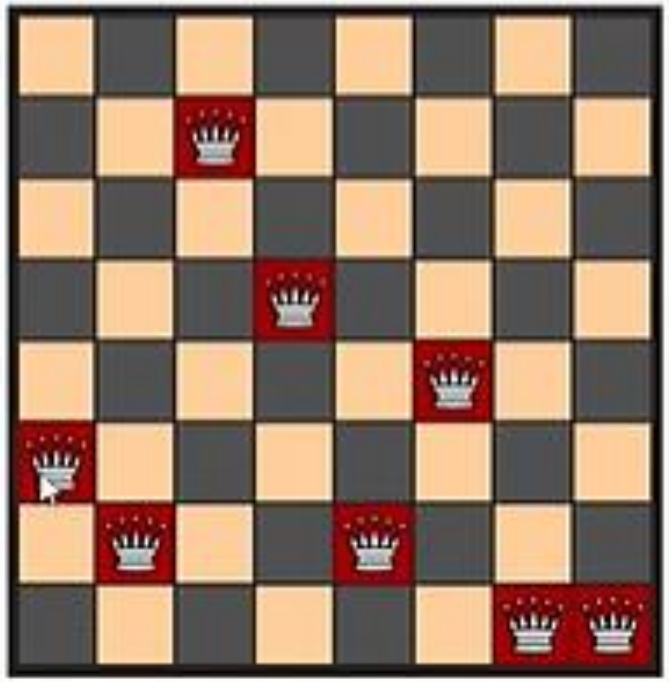
Solving 8-Queens using Genetic Algorithm

- Step 1: Representing individuals.
- Step 2: Generating an initial Population.
- Step 3: Applying a Fitness Function.
- Step 4: Selecting parents for mating in accordance to their fitness.
- Step 5: Crossover of parents to produce new generation.
- Step 6: Mutation of new generation to bring diversity.
- Step 7: Repeat until solution is reached.

Step 1: Representing Individuals/States

- Formulate an appropriate method to represent individuals of a population.
- Array.
- Index: Column.
- Value: Row.

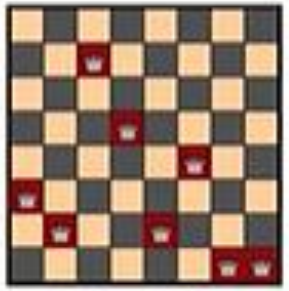
3	2	7	5	2	4	1	1



Step 2: Generate Initial Population

- Generate random arrangements of 8 queens on a standard chess board.

A



3	2	7	5	2	4	1	1

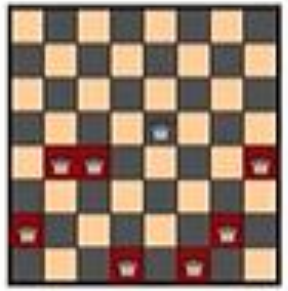
B

2	4	7	4	8	5	5	2

C

3	2	5	4	3	2	1	3

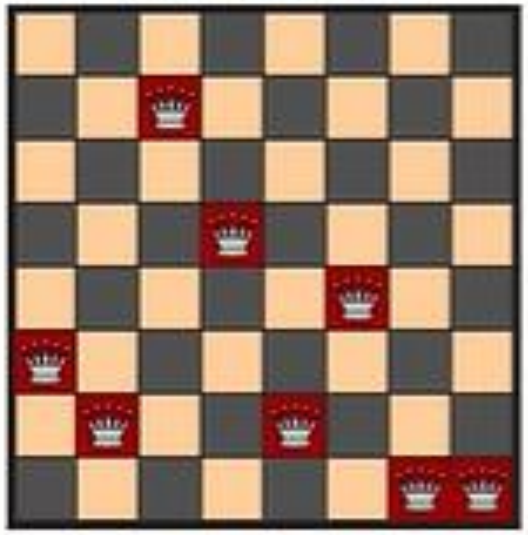
D



2	4	4	1	5	1	2	4

Step 3: Apply Fitness Function

Individual



3	2	7	5	2	4	1	1

Fitness = No. of non attacking pairs

- **Queen 1:** 6
- **Queen 2:** 5
- **Queen 3:** 4
- **Queen 4:** 3
- **Queen 5:** 3
- **Queen 6:** 2
- **Queen 7:** 0
- **Queen 8:** 0

Total 23

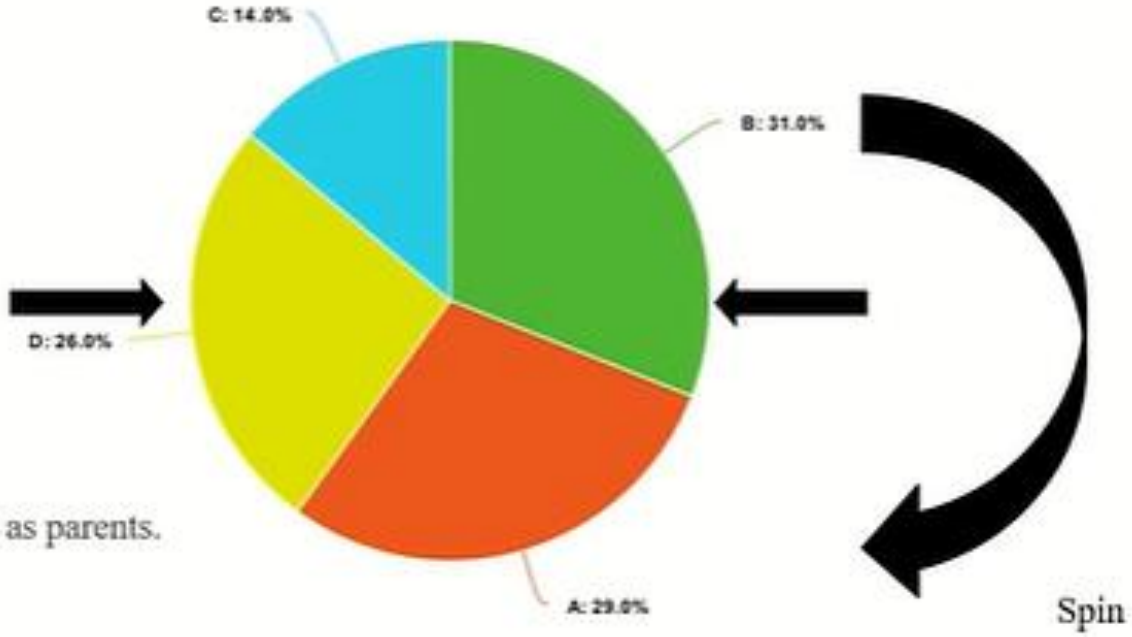
Step 3: Apply Fitness Function (contd.)

	A	B	C	D
Individuals				
	3 2 7 5 2 4 1 1	2 4 7 4 8 5 5 2	3 2 5 4 3 2 1 3	2 4 4 1 5 1 2 4
Fitness	23	24	11	20
Fitness %	29%	31%	14%	26%

Total:78

Step 4: Selection

Proportionate selection or roulette wheel selection gives individuals **with higher fitness values** a higher chance of being selected as parents, mimicking the concept of "survival of the fittest" in the natural evolutionary process.

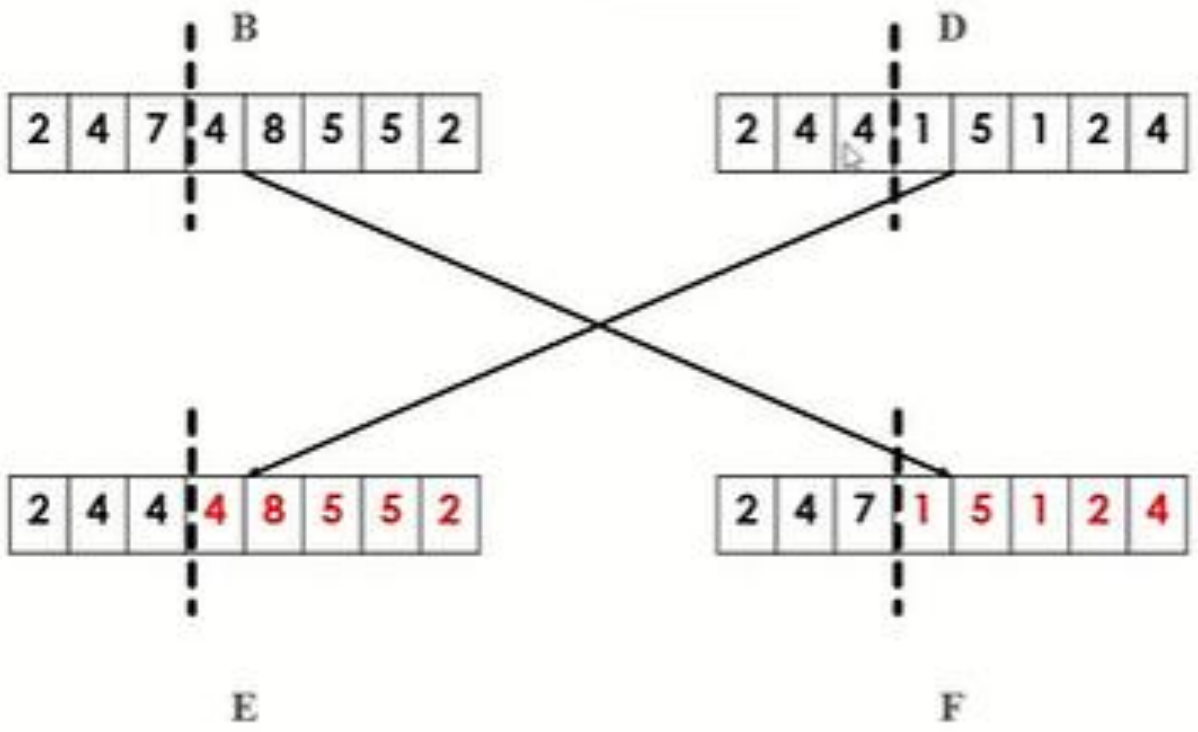


- B and D are selected as parents.

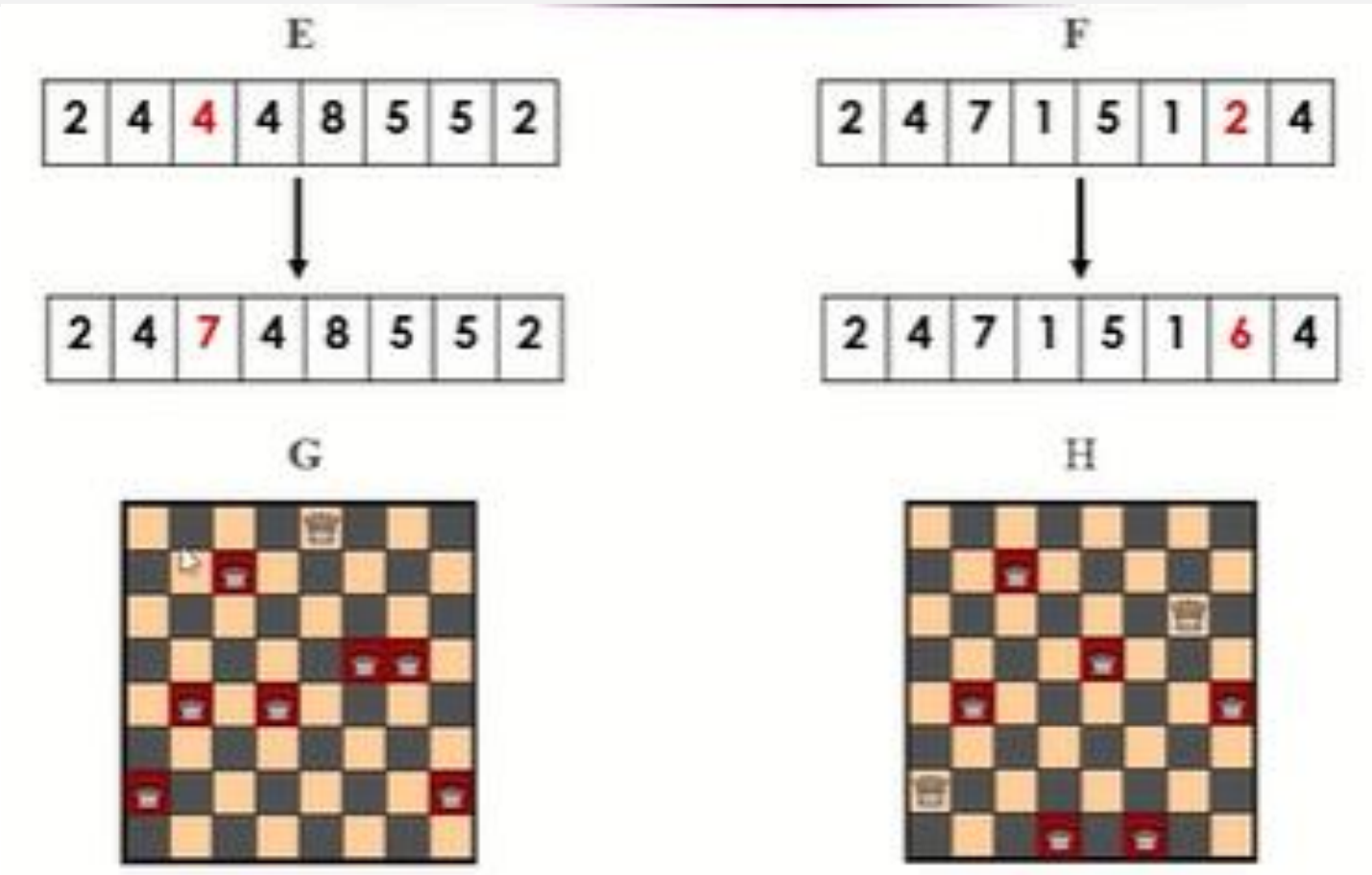
- There are various methods of selection.
- Roulette Wheel, Tournament, Rank, etc.
- Stochastic Universal Sampling (SUS).
- Population is divided on a wheel according to their respective percentages of fitness and two fixed points are placed.
- Wheel is spun and those individuals are selected at which the fixed points are pointing when the wheel stops.

The key difference between SUS and traditional roulette wheel selection is that **SUS selects multiple parents in a single pass, rather than selecting one parent at a time.**

Step 5: Crossover



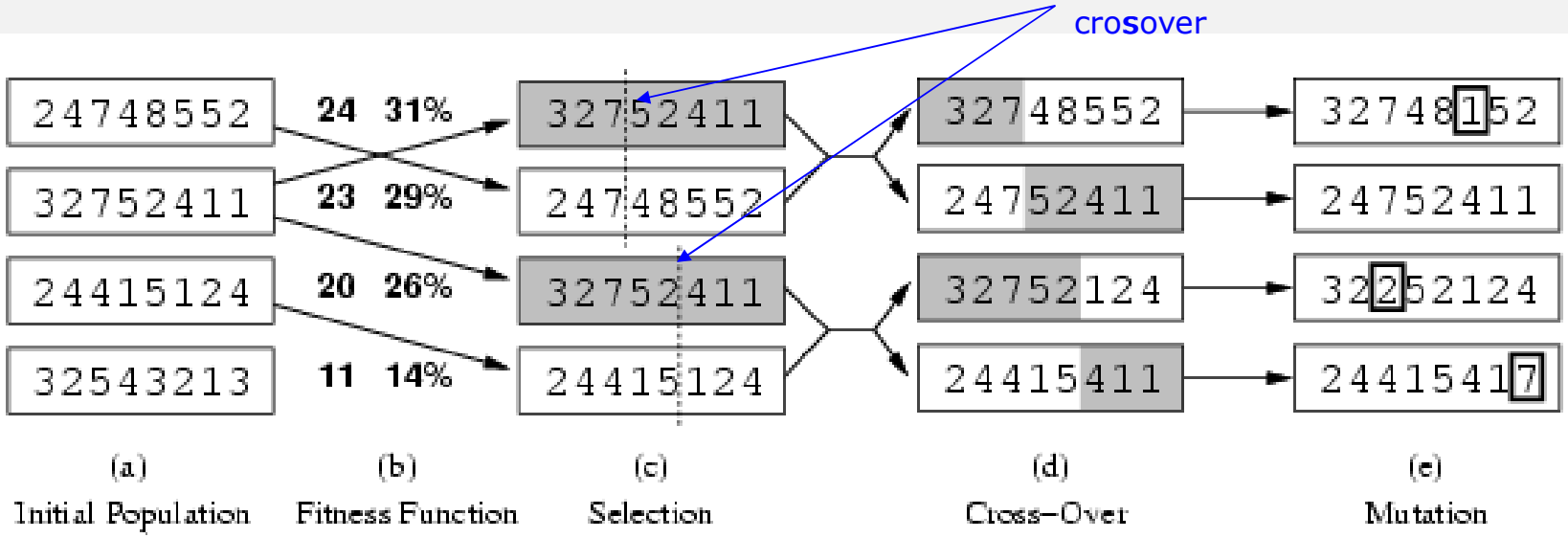
Step 6: Mutation



Step 7: Repeat

- All steps are repeated until best solution is reached.
- Best solution = Highest fitness score (28 in this case).

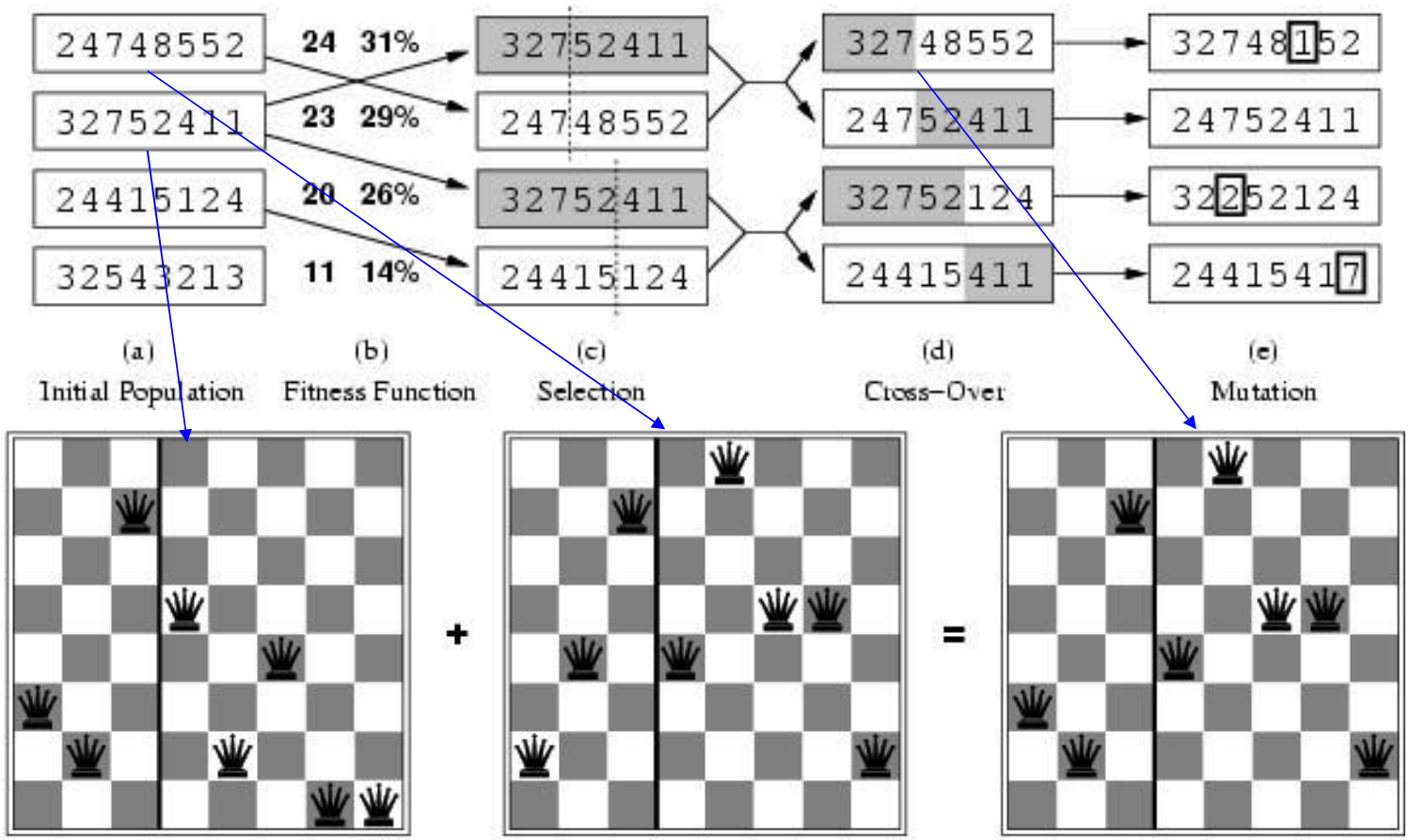
Genetic algorithms



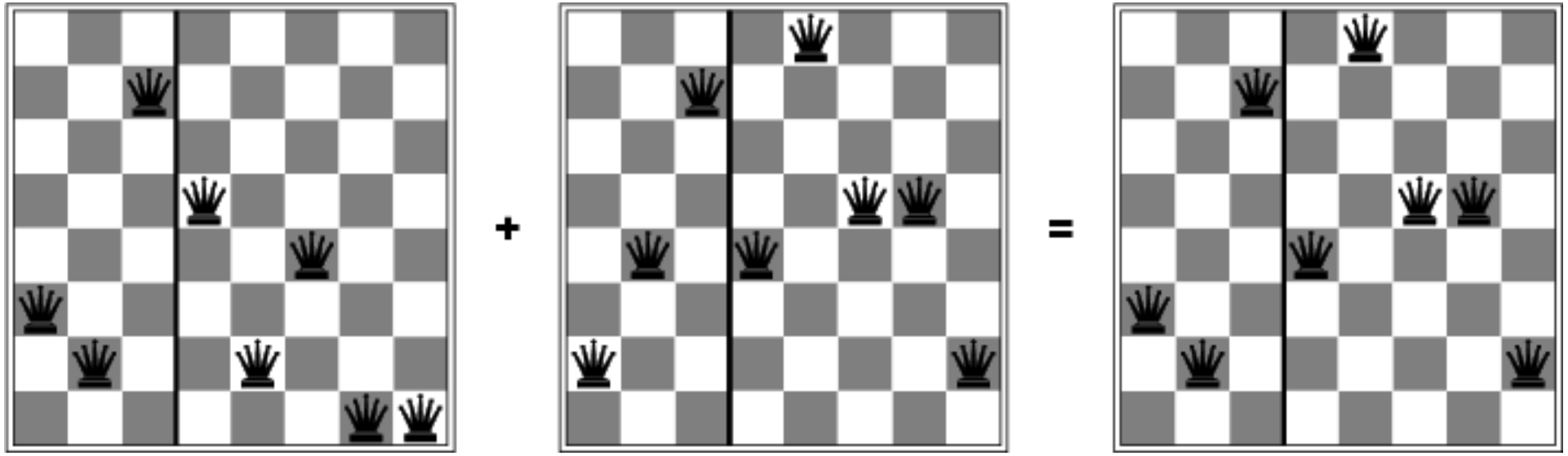
Example 8-Queen

- Fitness function: number of non-attacking pairs of queens (min = 0, max = 28)
- 24/(24+23+20+11) = 31%
- 23/(24+23+20+11) = 29% etc

8-Queen Example



Genetic algorithms



Algorithm

```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
  inputs: population, a set of individuals
          FITNESS-FN, a function that measures the fitness of an individual

  repeat
     new_population ← empty set
     for i = 1 to SIZE(population) do
        x ← RANDOM-SELECTION(population, FITNESS-FN)
        y ← RANDOM-SELECTION(population, FITNESS-FN)
        child ← REPRODUCE(x, y)
        if (small random probability) then child ← MUTATE(child)
        add child to new_population
     population ← new_population
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to FITNESS-FN
```

```
function REPRODUCE(x, y) returns an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, c + 1, n))
```

Figure 4.8 A genetic algorithm. The algorithm is the same as the one diagrammed in Figure 4.6, with one variation: in this more popular version, each mating of two parents produces only one offspring, not two.

Applications: Genetic algorithms

- DNA Analysis
- Image Processing
- Vehicle Routing
- Neural Network

Solving Difficult Problems

GAs prove to be an efficient tool to provide **usable near-optimal solutions** in a short amount of time.

Failure of Gradient Based Methods

In most real-world situations we have a very complex problem as it suffer from an inherent tendency of getting stuck at the local optima as shown here.

Getting a Good Solution Fast

Some difficult problems like the Travelling Salesperson Problem (TSP), have real-world applications like path finding and VLSI Design.

Solving 8-Queens using Genetic Algorithm

- Step 1: Representing individuals.
- Step 2: Generating an initial Population.
- Step 3: Applying a Fitness Function.
- Step 4: Selecting parents for mating in accordance to their fitness.
- Step 5: Crossover of parents to produce new generation.
- Step 6: Mutation of new generation to bring diversity.
- Step 7: Repeat until solution is reached.

Example: Knapsack Problem (Capacity=12kg)

Item	Weight (kg)	Value
A	5	10
B	3	5
C	7	9
D	2	8

Problem is that which item should be kept in the knapsack so as it will maximize knapsack value without breaking knapsack

Knapsack Problem

*0 represents absence of item
*1 represent the presence of item
State Space=2^4

Population

A	B	C	D
0	1	1	0
0	0	1	1
0	1	1	1
1	0	1	0

Fitness

Fitness Function for 1:
Weight=3+ 7=10
Value=5+9=14

Fitness Function for 2:
Weight=7+ 2=9
Value=8+9=17

Fitness Function for 3:
Weight=3+ 7+2=12
Value=5+9+8=22

Fitness Function for 4:
Weight=5+ 7=12
Value=10+9=19

Selection

0	1	1	1
0	0	1	1
1	0	1	0
0	1	1	1

Knapsack Problem

*0 represents absence of item
*1 represent the presence of item
State Space=2^4

Population

	A	B	C	D
C1	0	1	1	0
C2	0	0	1	1
C3	0	1	1	1
C4	1	0	1	0

Fitness

Fitness Function for 1:
Weight=3+ 7=10
Value=5+9=14

Fitness Function for 2:
Weight=7+ 2=9
Value=8+9=17

Fitness Function for 3:
Weight=3+ 7+2=12
Value=5+9+8=22

Fitness Function for 4:
Weight=5+ 7=12
Value=10+9=19

Selection

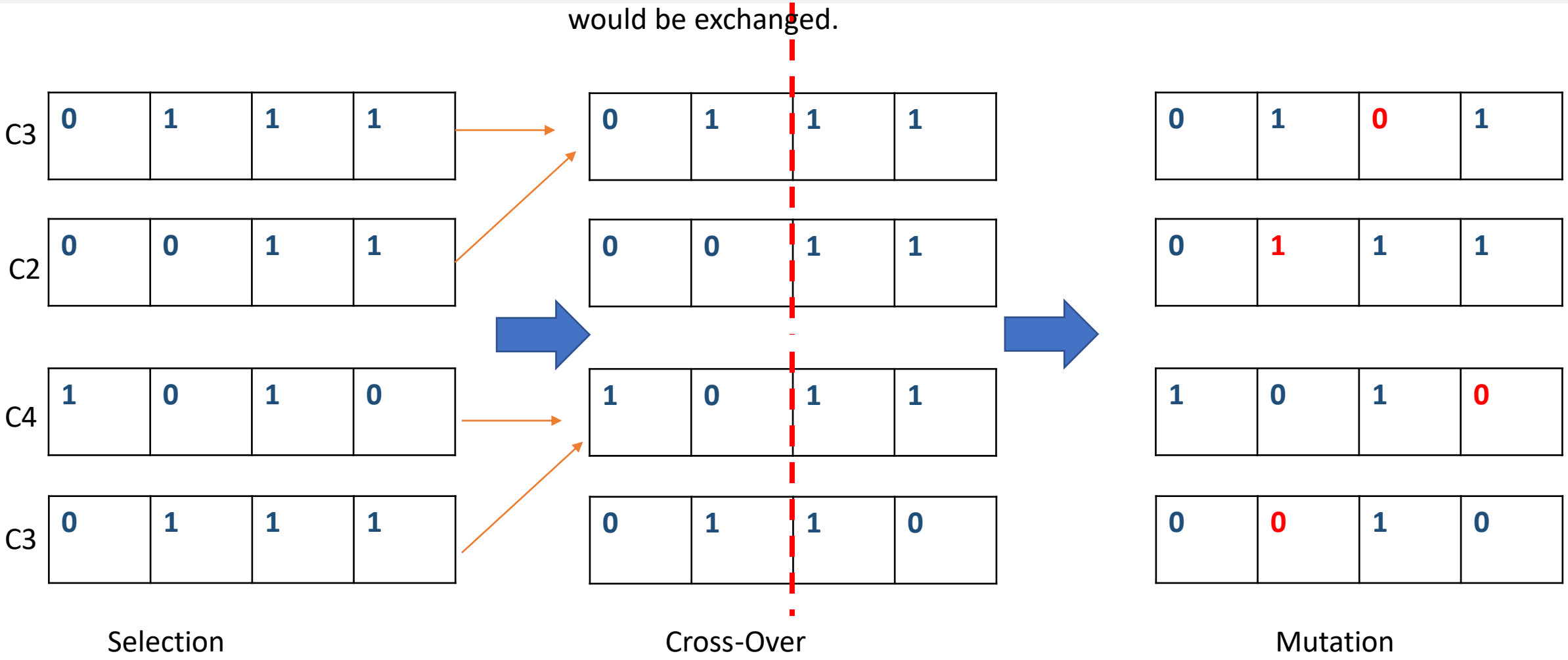
Roulette Wheel selection: the state that has the highest fitness value gets larger share of the wheel

0	1	1	1	C3
0	0	1	1	C2
1	0	1	0	C4
0	1	1	1	C3

Knapsack Problem

*0 represents absence of item
*1 represent the presence of item
State Space=2^4

Crossover: Randomly select the position around which gene would be exchanged.



Knapsack Problem

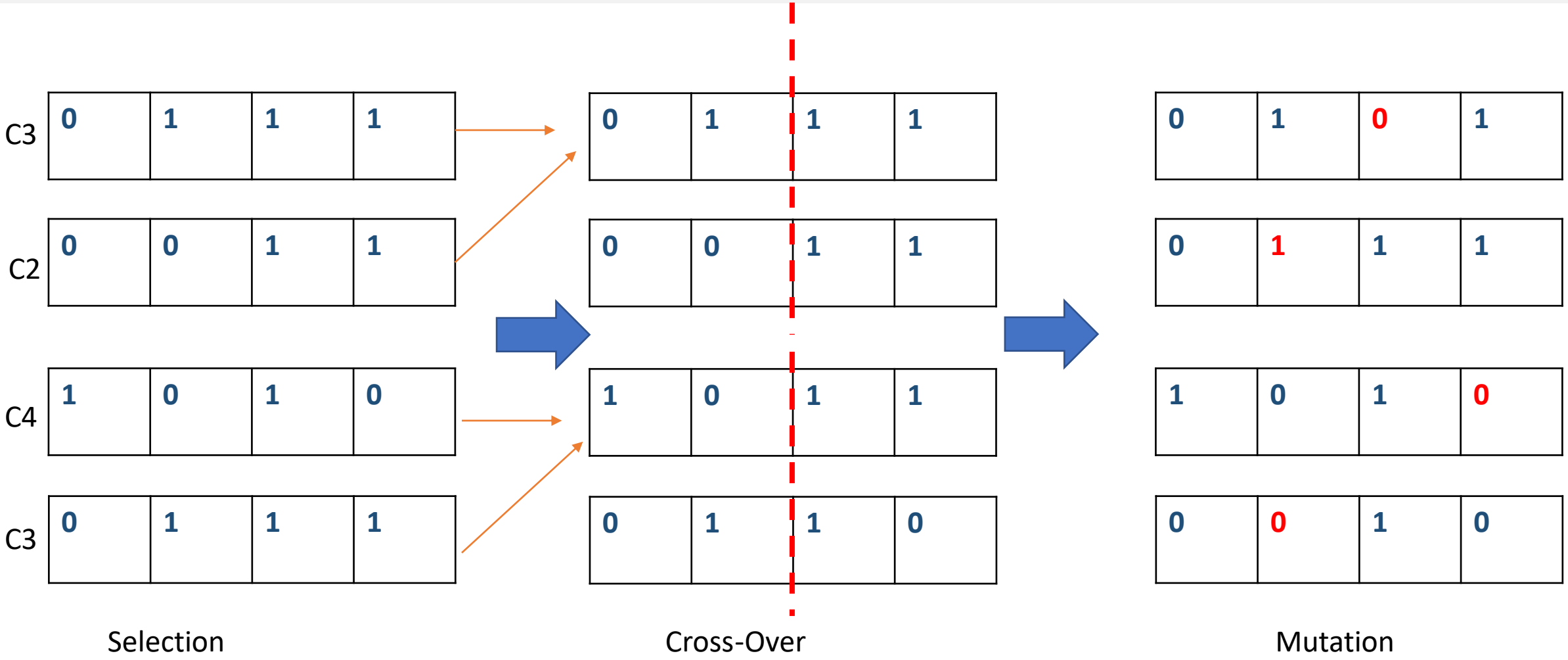
*0 represents absence of item
*1 represent the presence of item
State Space=2^4

Introduces the diversity within the population so that search algorithm does not necessarily get stuck at local maxima.

Mutation

Knapsack Problem

*0 represents absence of item
*1 represent the presence of item
State Space=2^4



Genetic algorithms

- Genetic algorithms combine an uphill tendency with random exploration and exchange of information among parallel search threads
- Advantages come from “crossover”, which raise the level of granularity
- Does not require any derivative information which may not be available for many real-world problems
- Gives n no of Solution not just a single solution
- Useful when search space is very large

GENETIC ALGORITHM

5. Selection Algorithm

It is used to select parents from the current population to be mated with one another to create children

The selection is primarily based on the fitness. The better the fitness of a chromosome, the greater its chance of being selected to be a parent

The rate at which a selection algorithm selects individuals with above average fitness is *selective pressure*

If there is not enough *selective pressure*, the population will fail to converge upon a solution. If there is too much, the population may not have enough diversity and converge prematurely

2

5. Selection Algorithm

Basically four types of selection algorithms:

- **random selection**
- **proportionate selection**
- **linear rank selection**
- **tournament selection**

Selection algorithm

Random Selection

Individuals are selected randomly with no reference to fitness at all

All the individuals, good or bad, have an equal chance of being selected

Selection Algorithm

Proportionate selection

Chromosomes are selected based on their fitness relative to the fitness of all other chromosomes

For this all the fitness are added to form a sum S and each chromosome is assigned a relative fitness (which is its fitness divided by the total fitness S)

A process similar to spinning a roulette wheel is adopted to choose a parent; the better a chromosome's relative fitness, the higher its chances of selection

Selection Algorithm

The selection of only the most fittest chromosomes may result in the loss of a correct gene value which may be present in a less fit member (and then the only chance of getting it back is by mutation)

One way to overcome this risk is to assign probability of selection to each chromosome based on its fitness

In this way even the less fit members have some chance of surviving into the next generation

Selection Algorithm

The probability of selection of a chromosome "i" may be calculated as

$$p_i = \text{fitness}_i / \sum_j \text{fitness}_j$$

Example

Chromosome	Fitness	Selection Probability
1	7	7/14
2	4	4/14
3	2	2/14
4	1	1/14

-

5. Selection Algorithm

Advantage

Selective pressure varies with the distribution of fitness within a population. If there is a lot of fitness difference between the more fit and less fit chromosomes, then the selective pressure will be higher

Disadvantage

As the population converges upon a solution, the selective pressure decreases, which may hinder the GA to find better solutions

5. Selection Algorithm

Linear rank selection

Even after selecting chromosomes by their probabilities there is a risk that a chromosome may have a vital gene but its fitness may be zero and hence its selection probability will be zero

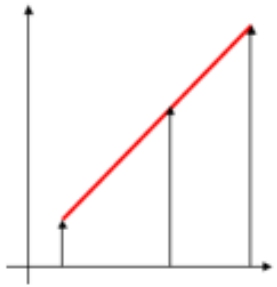
The linear rank method assigns a small non-zero probability to even those chromosome which have a zero fitness

Linear rank selection

The population is sorted from best to worst according to the fitness

Each chromosome is then assigned a new fitness based on a *linear* ranking function

New Fitness = (P – r) + 1



where P = population size, r = fitness rank of the chromosome

If P = 11, then a chromosome of rank 1 will have a New Fitness of 10 + 1 = 11 & a chromosome of rank 6 will have 6

Linear rank selection

A user adjusted slope can also be incorporated

$$\text{New Fitness} = (P - r)\,(\max - \min)/(P - 1) + \min$$

where max and min are set by the user to determine the slope (max - min)/(P – 1) of the function

Let P = 11, max = 8, min = 3,
then a chromosome of rank 1 will have a New fitness of 10*5/10 + 3 = 8
& a chromosome of rank 6 will have 5*5/10 + 3 = 5.5

Linear rank selection

Once the new fitness is assigned, parents are selected by the same roulette wheel procedure used in proportionate selection

Advantage: selective pressure, once determined by the user, remains constant

Disadvantage:
- **Population must be sorted**
- **Chromosomes with the same fitness will not have the same probability of being selected**

- Another way of linear rank selection is:

- The highest ranking chromosome has probability *p* of selection (e.g. 0.66)

- If the 1st chromosome fails to get selected, then it is taken out of competition and the second ranking chromosome gets the chance of selection with a probability *p* among the remaining chromosomes (hence its overall probability is 0.33 x 0.66 = 0.22)

- This process is continued for all the chromosome until one of them gets selected

Example

Chromosome	Fitness	Proportion Selection
1	7	7/14 = 0.5
2	4	4/14 = 0.28
3	2	2/14
4	1	1/14
5	0	0/14 = 0

5. Selection Algorithm

Tournament selection

One parent is selected by comparing a subset *b* of the available chromosomes, and selecting the fittest; a second parent may be selected by repeating the process

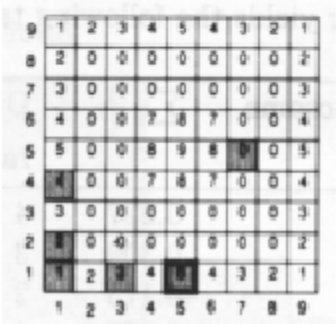
The selection pressure increases as *b* increases

Value of *b* = 2 is most commonly used

Survival of the most diverse

If a chromosome is close to the best chromosome and has a higher score than a chromosome that is far away (diverse) but has a slightly lower score, then the far away chromosome will be wiped out

This results in uniformity, even in large populations



17

Survival of the most diverse

Diversity of a chromosome from the already selected chromosomes can be calculated as

- **Take the distance of the chromosome from a selected chromosome**
- **Square the distance (to remove negative distance)**
- **Take the sum of all squared distances**
- **The larger this sum the more diverse is the chromosome**

$$\text{Diversity} = \sum_i \text{distances}_j^2$$

One way to incorporate diversity is to rank the chromosomes by the sum of quality rank and diversity rank

Survival of the most diverse

Hence diversity (or being different) and can be incorporated into the selection procedure and a chromosome which is almost as good as another chromosome but is far away from the already selected chromosomes has a better chance of survival

Homework

- Readings
  - CH 4 Section 4.1